

TRABAJO PRÁCTICO

Visualización de algoritmos de ordenamiento

Curso: Introducción a la programación

Comisión: 10

Integrantes del grupo:

- Gabriel Segovia
- Ivan Vazquez

Introducción

El objetivo de este trabajo es implementar algoritmos de ordenamiento utilizando Python para que ordenen los elementos de una lista representada visualmente a través de un gráfico de barras en una interfaz web proporcionada por los profesores. Mediante una función llamada “step” que permite ver el proceso cada vez que sucede un micropaso, se implementaron los algoritmos Bubble, Insertion, Selection, Quick y Shell en cada archivo python respectivamente. En este informe se dará una explicación de cada algoritmo y decisiones llevadas a cabo durante su implementación.

Bubble Sort

El bubble sort es un algoritmo de ordenamiento con una estructura simple, el cual ordena como su nombre indica, burbujeando los elementos hacia su posición. Este algoritmo usa bucles for con rangos de variable n que funciona como la cantidad de elementos dentro de una lista, que es recorrida en un for i in range por una variable i que sirve como comparador e incrementador entre los elementos de la lista que son ordenados, por último; el bucle es recorrido en un for j in range donde j sirve como índice y la variable con la que se hace el swap. Fueron determinadas en este trabajo, las variables a y b como j y j+1, lo que sirve para comparar y generar el swap entre los elementos de la lista.

Un problema general que hubo durante la realización del bubble sort aplicado en este trabajo fue el nulo uso de bucles. Esta decisión fue llevada a cabo cuando el algoritmo no respondía a pesar que incluso dentro del propio código el algoritmo funcionase correctamente. El problema fue resuelto cuando los bucles se intercambiaron por condicionales, lo que acertaba más a ser un micro-paso. Un único error resaltado fue el posicionamiento del condicional que finalizaba el algoritmo, el cual posteriormente fue corregido.

Código de Bubble Sort:

```
def init(vals):
```

```
    global items, n, i, j
```

```
    items = list(vals)
```

```
    n = len(items)
```

```
    i = 0
```

```
    j = 0
```

```
def step():
```

```
    global items, n, i, j
```

```
    if i >= n - 1:
```

```
        return {"done": True}
```

```
    a = j
```

```
    b = j + 1
```

```
    swap = False
```

```
    if items[a] > items[b]:
```

```
        swap = True
```

```
        items[a], items[b] = items[b], items[a]
```

```
    j += 1
```

```
    if j >= n - 1 - i:
```

```
        i += 1
```

```
        j = 0
```

```
    return {"a": a, "b": b, "swap": swap, "done": False}
```

Insertion Sort

El insertion sort, es un algoritmo que, en palabras generales, sostiene un elemento hasta que este esté en su posición correcta, esto lo hace mediante un desplazamiento del elemento hacia la izquierda hasta que no haya números mayores por detrás. Este algoritmo está compuesto de variables j , i , n , donde funcionan como índice, variable de intervalo y número de elementos de una lista respectivamente. En el algoritmo general se utiliza un `for i in range`, lo que permite recorrer la lista libremente, dentro de ese bucle `for`, se encontraba un bucle `while` que es el encargado del intercambio lateral de los elementos donde si $j-1$ es mayor a j , estos se desplazan atrás en la lista. En este trabajo las indicaciones exigían el uso de condicionales, por lo que en vez de un bucle `for`, la variable j comienza con valor `None`, lo que inicia el desplazamiento pero sin devolver un swap, posteriormente se genera un `while` donde se activa el swap y se intercambian los elementos, con j retrociendo a través de un $j=j-1$ para que el desplazamiento acabe únicamente cuando el valor de j sea mayor a $j-1$, posteriormente, la variable i se incrementa, la variable j vuelve a ser `None` y se retornan los valores.

En general no hubo problemas en la creación del código, detalles como un mal uso de las variables a y b que al final no debían ser incluidas. Un error destacable y molesto, fue el `return` del `while`, que no funcionaba como debería; como solución, se ingresó en una variable `out` los valores que debía retornar al finalizar el bucle `while`, luego de eso el código funcionaba correctamente

Codigo de Insertion Sort:

```
items = []  
n = 0  
i = 1    # empezamos desde el segundo elemento  
j = None
```

```

def init(vals):
    global items, n, i, j
    items = list(vals)
    n = len(items)
    i = 1
    j = None

def step():
    global items, n, i, j
    # - Si i >= n: devolver {"done": True}.
    if i >= n:
        return {"done": True}

    # - Si j es None: empezar desplazamiento para el items[i] (p.ej., j = i) y devolver un
highlight sin swap.
    if j == None:
        j = i
        return {"a": j, "b": j, "swap": False, "done": False}

    # - Mientras j > 0 y items[j-1] > items[j]: hacer UN swap adyacente (j-1, j) y devolverlo
con swap=True.
    while j > 0 and items[j-1] > items[j]:
        items[j-1], items[j] = items[j], items[j-1]
        out = {"a": j-1, "b": j, "swap": True, "done": False}
        j=j-1
        return out

    # - Si ya no hay que desplazar: avanzar i y setear j=None.
    i =i+1
    j = None
    return {"a": i, "b": i, "swap": False, "done": False}
    return {"done": True}

```

Selection Sort

Es un algoritmo que busca en la lista el valor mínimo y, luego de encontrarlo, lo coloca al principio de la lista. El proceso se repite con el siguiente valor más pequeño hasta que la lista queda completamente ordenada. La dificultad de este algoritmo recae en que hay que dividirlo en dos fases, una fase de búsqueda donde se recorre la lista no ordenada hasta encontrar el valor mínimo y guardarlo; y una fase de “swap” donde intercambian posiciones el valor mínimo encontrado con el valor de la posición del extremo izquierdo de la lista no ordenada.

Código de Selection Sort:

```
items = []
n = 0
i = 0      # cabeza de la parte no ordenada
j = 0      # cursor que recorre y busca el mínimo
min_idx = 0 # índice del mínimo de la pasada actual
fase = "buscar" # "buscar" | "swap"

def init(vals):
    global items, n, i, j, min_idx, fase
    items = list(vals)
    n = len(items)
    i = 0
    j = i + 1
    min_idx = i
    fase = "buscar"
```

```

def step():
    global items, n, i, j, min_idx, fase
    for i in range(i,n+1):
        for j in range(j,n+1):
            if fase=="buscar" and i!=n:
                if j==n:
                    fase="swap"
                    return {"swap": False, "done": False}
            if items[min_idx]>items[j]:
                min_idx=j
                j_actual=j
                j=j+1

        return {"a": min_idx, "b": j_actual, "swap": False, "done": False}

    if fase=="swap":
        a=min_idx
        b=i
        if min_idx!=i:
            a=i
            b=min_idx
            items[a], items[b] = items[b], items[a]
        i=i+1
        j=i+1
        min_idx=i
        fase="buscar"
        return {"a": a, "b": b, "swap": True, "done": False}

    if items[i]==items[n]:
        return {"done": True}

```

Quick Sort

Es un algoritmo que toma el principio de “divide y vencerás”, donde se divide el problema en subproblemas más pequeños, los resuelve de forma recursiva y combina sus soluciones para llegar a la solución del problema inicial. Es decir, divide el arreglo en dos y procede a ordenar las dos mitades aplicando el algoritmo sobre ambas. Una vez ordenados los subarreglos, los une.

La dificultad de este algoritmo es que es necesaria la elección de un elemento que sirva de pivote, ya que al momento de la partición, los elementos menores al pivote quedarán a su izquierda en un subarreglo y los mayores o iguales quedarán a su derecha en otro. Luego se aplica el mismo proceso a ambos subarreglos hasta que quede un solo elemento o ninguno.

Para poder aplicarlo, se decidió dividir el algoritmo en 4 fases:

- Una donde ocurre la partición.
- Otra donde los elementos que cumplan la condición se desplacen a un lado del pivote.
- La siguiente, donde se cambia el pivote para mantener la efectividad del algoritmo.
- Por último, la que añade los subarreglos izquierdo y derecho que quedan en la pila.

Código de Quick Sort:

```
items = []
n = 0
stack = [] # pila
inicio = 0
fin = 0
i = 0
j = 0
pivote = None
pivote_idx = None
fase = "particion" # "particion" | "buscarSwap" | "swapPivote" | "añadir"
```

```
def init(vals):
    global items, n, stack, inicio, fin, i, j, pivote, pivote_idx, fase
    items = list(vals)
    n = len(items)
    stack = [(0, n - 1)]
```



```
inicio = 0
fin = 0
i = 0
j = 0
pivote = None
pivote_idx = None
fase = "particion"
```

```
def step():
```

```
    global items, n, stack, inicio, fin, i, j, pivote, pivote_idx, fase
```

```
    if n < 2:
```

```
        return {"done": True}
```

```
    if fase == "particion":
```

```
        if not stack:
```

```
            return {"done": True}    # Finaliza cuando ya no hay subarreglos
```

```
        inicio, fin = stack.pop()
```

```
        if inicio >= fin:            # el subarreglo es trivial
```

```
            return {"done": False}
```

```
        # inicializamos la partición
```

```
        pivote = items[fin]
```

```
        i = inicio - 1
```

```
        j = inicio
```

```
        fase = "buscarSwap"
```

```
        return {"done": False}
```

```
    if fase == "buscarSwap": #Se recorre con J y se realiza el swap si es necesario
```

```
        if j < fin:
```

```
            if items[j] <= pivote:
```

```
                i = i + 1
```

```
                a = i
```

```

    b = j
    items[a], items[b] = items[b], items[a]
    j = j + 1
    return {"a": a, "b": b, "swap": True, "done": False}
else:
    j = j + 1
    return {"swap": False, "done": False}

```

fase = "swapPivote" #Finaliza el recorrido, cambio de fase

if fase == "swapPivote": #Pone el pivote en su lugar

```

    a = i + 1
    b = fin
    items[a], items[b] = items[b], items[a]
    pivote_idx = a
    fase = "añadir"
    return {"a": a, "b": b, "swap": True, "done": False}

```

if fase == "añadir": #Se añaden los subarreglos restantes a la pila

```

    # subarreglo izquierdo
    if pivote_idx - 1 > inicio:
        stack.append((inicio, pivote_idx - 1))

```

```

    # subarreglo derecho
    if pivote_idx + 1 < fin:
        stack.append((pivote_idx + 1, fin))

```

```

    fase = "particion"
    return {"swap": False, "done": False}

```

return {"done": False}

Shell Sort

El algoritmo compara y ordena los elementos en sublistas, separadas por una “brecha”/”salto” (gap). El gap se va reduciendo a medida que listas se van ordenando hasta quedar en gap=1 dónde ocurre un Insertion Sort sobre una lista casi ordenada.

No se encontraron complicaciones al implementar este algoritmo más allá de determinar cuándo hacer el cambio de fases y reducir el gap. Una vez dentro de la fase swap, simplemente se aplica Insertion Sort.

Código de Shell Sort:

```
items = []
n = 0
gap = 0
i = 0
j = 0
Fase = "buscar" # "buscar" | "swap"
```

```
def init(vals):
    global items, n, gap, i, j, Fase
    items = list(vals)
    n = len(items)
    gap = n // 2 #Genera el primer gap
    i = gap
    j = 0
    Fase = "buscar"
```

```
def step():
    global items, n, gap, i, j, Fase
```

```
    if Fase == "buscar": # Recorremos i desde gap hasta n-1
        if i >= n:
```

```

    gap = gap // 2 # Nuevo gap
    if gap == 0:

        return {"done": True}
    i = gap
    Fase = "buscar"
    return {"swap": False, "done": False}

j = i # Inicializamos j para la siguiente fase
Fase = "swap"
return {"swap": False, "done": False}

if Fase == "swap":
    a = j
    b = j - gap

    if b >= 0 and items[a] < items[b]:

        items[a], items[b] = items[b], items[a]

        j = b # Necesario para que se siga desplazando
        return {"a": a, "b": b, "swap": True, "done": False}

i = i + 1
Fase = "buscar" #Si no hay swap, volvemos a la fase de búsqueda.

return {"a": a, "b": b, "swap": False, "done": False}

```